職務経歴書

最終更新: 2026年10月30日

基本情報

項目	内容
氏名	菊地 弘祐 (きくち こうすけ)
生年月	1984年3月
居住地	兵庫県西宮市
最終学歴	早稲田大学大学院 創造理工学研究科 建築学専攻 博士課程単位取得退学
GitHub	https://github.com/kousukekikuchi1984

職務要約

2014年からバックエンド、データ基盤、インフラを横断して開発しているソフトウェアエンジニアです。 会計データ、ポイント残高、広告収益など、誤りが確定すると修正コストが大きい領域で、 冪等性、状態遷移、再実行、監査可能性を含むデータ整合性設計を行ってきました。

現在は株式会社NTT-MEのAI建物レジリエンス事業で、事業移管後のシステム把握とプラットフォーム改善を担当しています。 ICとして、要求の明確化、アーキテクチャ、データ、インフラ、セキュリティ、運用を横断しています。 サーバーレスDWHの復元、Terraformの構成管理の正常化に加え、役割別AI Agent、統合テスト、 検証用テストレーン、依存ライブラリ更新・修正の自動化を実務で検証しています。 人間が全体整合性と設計判断に集中できるSoftware Factoryを段階的に構築しています。 最近は、Evidenceと変更影響範囲に基づく自動承認CIを設計・実装し、本格運用に向けて準備しています。 セキュリティ修正の品質ゲートを定め、継続的な修正・検証の仕組みも構想しています。 Dependabot Reachabilityを導入して脆弱性情報の取得を確認し、技術検証と業務上の受け入れを分ける リリース戦略を整理しました。Blue/Greenは構想段階で、導入・運用効果は未検証です。

AIに作業を任せる際は、仕様、制約、完了条件を先に定め、生成物を評価する基準と人間が判断する範囲を設計しています。 役割別Agentの局所最適による過剰設計を観測し、共有情報、依存関係、Agent間の調整を設計課題として整理しています。 個人の実装速度だけでなく、同僚の工数、変更を受け入れる準備、運用負荷を考慮して改善を進めています。

得意領域

- データ整合性設計: 冪等性、状態遷移、ロック順序、境界条件、再実行、監査可能性。
- バックエンド・プラットフォーム設計: 可用性、コスト、運用性、変更容易性を踏まえた設計判断。
- 障害の根本原因の特定: アプリケーション、データベース、インフラ、運用を横断した調査と構造的な修正。
- データ基盤・ワークフロー: データ抽出、集計、検証、確定ルール、リトライ、異常検知、復旧の設計。
- 要求の明確化: 曖昧な依頼を、対象範囲、依存関係、リスク、受け入れ条件が明確な仕様へ変換。
- Software Factory: 仕様駆動開発、役割別Agent、生成コードのレビュー、検証基盤、依存更新の自動化と人間による設計判断。
- 承認・修正プロセスの設計: Evidence、変更影響範囲、修正完了条件、権限・責任に基づく品質ゲート。

技術領域

領域	技術・経験
バックエンド	Python（主な業務言語、内部実装理解、メモリプロファイリング、非同期処理）。Rust、Java、Goは限定的な経験。
フロントエンド	TypeScript、Vue.js（業務利用）。Flutterは限定的な経験。
データベース	MySQL（性能・運用）、DynamoDB、OpenSearch、DuckDB、Athena。
AWS インフラ	ECS、EKS、Lambda、S3、CloudFront、CloudWatch、EventBridge、Step Functions、Terraform。
データ処理	Digdag、Embulk（運用、トラブルシュート、実装解析）、DWH、バッチ処理。
Software Factory	Cursor、LLM、仕様駆動開発、コードベース探索、役割別Agent、コードレビュー、統合テスト、テストレーン分離、依存更新・修正の自動化。
その他	Linuxのプロファイリング・運用調査、HULFTによるファイル連携、Dependabotによる依存ライブラリ更新。

職歴

株式会社NTT-ME / Software Engineer

2026年6月–現在

所属: 社会インフラデザイン部 地域あんしん推進部門 防災担当 AI建物レジリエンス事業。 英国Tractable社から譲渡された事業で、PdM、TL兼PMと連携するICとしてシステムの把握と改善を担当しています。 正式な役職はSoftware Engineerであり、Platformの設計・運用と技術課題の解決を担っています。

Evidenceと変更影響範囲に基づく自動承認CIを設計・実装

- 実装の高速化に対して承認待ちが制約となる状況を踏まえ、自動ApproveとHuman Reviewを振り分けるCIを設計・実装した。
- 指摘の重大度、変更影響範囲、必要なIntegration Test / E2EのEvidenceを承認条件として整理した。
- 自動Approve、自動Merge、本番反映を別の判断として扱い、人間の注意力を難しい判断へ配分する方針を定めた。
- 本格運用前であり、Review Lead Timeやレビュー待機時間の削減効果は未実証。

セキュリティ修正の品質ゲートと継続対応の仕組みを設計

- モジュール単位の修正、Integration Testの通過、EvidenceのPR添付を、修正の完了条件として明確にした。
- 実行環境のInventoryと判断Skillの整備に着手し、影響評価から修正、検証、承認、リリース、反映後確認をつなぐ構想を整理した。
- Security Remediation Factory全体の本格運用と、修正完了率や処理能力の改善効果は今後の検証対象。

Dependabot Reachabilityの導入と脆弱性評価の情報取得

- Dependabot Reachabilityを有効化し、読み取り専用アクセスで脆弱性情報を取得できることを確認した。
- 情報収集・評価を専用Agentへ分離し、修正・テストを担うAgentとの責任分界を整理した。
- Reachability不明を安全とも悪用可能とも決めつけず、本番の依存関係・実行経路と照合する方針を定めた。
- 継続運用の判断品質、対応時間、修正完了率への効果は未検証。

検証責任と変更影響に基づくリリース戦略の整理

- 技術的なVerificationと業務上の受け入れを分け、変更の影響と緊急度に応じてリリース経路を設計する方針案を整理した。
- Blue/Green Deploymentは構想段階。共有データ、イベントの二重処理、DB / APIの後方互換性、 ロールバックは今後の設計・検証課題であり、導入やリリース頻度の改善を実績には含めない。

仕様とテストを起点に**AI Agent**の検証基盤を整備

- 統合テストを実装し、Coding Agentが実装後に検証できる基盤を整備した。
- 検証用テストレーンを構築し、通常処理とリリース検証用の処理を分離した。
- Cursorがログや関連データを参照できる環境を整え、実装後の調査・検証を支援した。
- 別サービスへのテストレーン導入と統合テスト拡充は今後の取り組み。確認工数の削減効果は未計測。

依存ライブラリの更新・検証・修正を自動化

- Dependabotによる依存ライブラリ更新後にテストを実行し、失敗した場合はCursorが修正する工程を導入した。
- 反復作業をAgentへ委譲する範囲を広げ、人間が全体整合性と設計判断を担う開発・検証の仕組みを段階的に整えている。

サーバーレス**DWH**の復元

事業移管後に従来の分析基盤が利用できなくなり、複数CSVの集計に週1回、1〜2時間を要していた。 案件数とSLA処理時間を確認できるデータ基盤を、要求整理から設計、実装、検証まで一貫して担当した。

- DynamoDB Export、S3、EventBridge、Step Functions、Athenaで、常時稼働サーバーを持たないDWHを設計・実装。
- 要求整理と設計を含む全体は約1か月、実装・インフラ構築・データ検証は実働5営業日で完了。
- 件数、欠損、重複、処理時間を既存データと照合した。
- データ基盤の構築と検証は一旦完了。実利用による工数削減、SLA報告や意思決定への効果は今後の確認対象。

OpenSearch障害の原因特定と**Terraform**構成管理の是正

- OpenSearchのインスタンス容量不足を特定し、スペック変更のコスト、適用手順、Terraformとtfstateの整合方法を整理した。
- S3の状態不整合、Lambdaの恒常的な再作成、Cognitoの定義不足、EC2のAMI検索による不確定なplanを個別に修正した。
- 変更がなければTerraform planに差分が出ず、applyが正常終了する状態を確認した。
- DynamoDBからOpenSearchへのDLQ再処理は、versionによる競合制御と通知から始める導入方針を検討した。 この再処理方式は設計・検討段階である。

LLMを用いたレビューと役割別**Agent**の検証

- Cursorを利用するCI/CDレビュー環境を構築。単一の計測事例で、PRレビュー時間を約60分から約5分へ短縮した。
- 変更目的、実行経路、権限・データへの影響、障害時の挙動を自分で確認した上で最終判断した。
- 実装、障害調査、セキュリティ、SRE、QA、レビューを役割別Agentとして試行した。
- DWHでは同じ仕様から2〜3回独立に生成し、テストと実環境確認で機能的な再現性を検証した。検証範囲はDWH一件。
- Agentごとの局所最適による過剰設計を観測し、共有情報、依存関係、Agent間の調整、人間による全体整合性の確認を設計課題として整理した。

要求の明確化と短期間の開発

- 入社2週目半ばから3週目半ばまでの約1週間でPoC向けUI改修を完了。Cursorをコード探索と実装支援に利用した。
- 要件が不足したAWSセキュリティタスクの対象範囲と対応方針を明確にし、実行可能な仕事へ整理した。
- クライアントPoCでは実装前に関係者と要求をすり合わせ、手順と完了条件を持つタスクへ分解した。
- AWSのネットワーク構成変更について、ルーティング、VPC endpoint、DNS、IAM条件、各サービスへの影響を評価した。
- 改善の導入時期と範囲は、同僚の工数とプロジェクトの準備状況に合わせて調整している。

YUZURIHA / Engineer（バックエンド・プラットフォーム）

2021年1月–2026年5月

会員700万人規模のポイントプラットフォームで、ポイント残高、会計出力、外部カード連携を担当。 約10名のチームの技術方針・設計レビュー・コードレビューに関わり、3〜4名のサブプロジェクトをリードした。 エンジニア採用の一次面接も担当した。

外部データ連携: 30万件、1時間SLA

- クレジットカード会社からの30万件のデータを、1時間以内に安全に取り込む処理を設計した。
- 外部仕様の例外を整理し、カード単位のレコードからユーザー単位の投入キーとロック粒度を定義した。
- 検証と投入を分離し、冪等な再実行と、取り込み中のアカウント統合に対応する補正を実装した。
- 実測約6分で取り込みを完了。先方起因の障害10件、うち誤データ3件を投入段階で遮断し、不可逆な誤計上ゼロで運用した。

会計CSVの誤計上をリリース前に防止

- 月次集計と外部会計連携で使うCSVに、ポイント失効時の計上ルールと実装の不整合を発見した。
- 一時補正ではなく、計上ルールに合致する設計へ修正し、誤った会計データが確定する事態をリリース前に防いだ。

HULFT連携基盤の更新

- OS、通信アプリケーション、暗号方式の更新を、インフラ担当と外部関係者を含む4名で進めた。
- 外部の切替コストという新しい制約を受け、コスト・リスク・復旧可能性を再評価し、段階移行から一括移行へ方針を変更した。
- バックアップ、復旧手順、重点監視を整え、致命的な障害なく切替を完了した。

性能改善とデータ整合性の両立

- ポイント残高の整合性や復旧処理を複雑にする可能性のある局所的な最適化を止め、ボトルネックを調査した。
- 主な待ちがMySQLのcommit時のfsyncにあることを特定し、commit数削減と責務分離を代替案として提示した。
- データ整合性を優先する方針をチームで合意した。

本番障害の根本原因の特定と再発防止

- メモリリークでは、`functools.cache`がDB Engineを参照し続けることを特定。 オブジェクトの生成・再利用・破棄を見直し、メモリ使用量の継続増加を解消した。
- DBデッドロックでは、処理経路ごとにロック取得順序が異なることを特定。 常に同じテーブルからロックを取得する規約へ統一し、対象のデッドロックを解消した。
- レビューで前提、リスク、代替案を説明し、非同期処理・デプロイ・トランザクションの境界を共通の確認観点として整理した。
- 1on1、技術指導、日本語表現やドキュメントの支援を行い、判断理由を自分で説明できるようにすることを重視した。

その他: ポイントシステム刷新・運用、ネイティブアプリ向けサーバーサイド対応、派遣人員募集サイトの開発・更新。

Gunosy / Data Reliability Engineer

2019年5月–2020年12月

大規模データ基盤で、データ取得、移行、集計、業務運用を横断し、正しさと再現性を確認できるワークフローを設計した。

- SSP収益データを取得し、Athenaで分析可能な形式へ整備。 発生から60時間後の速報値確定、72時間後の最終確定という業務ルールを実装した。
- Digdagの挙動をコードから調査し、リトライによる重複実行などの問題を解析した。 収益按分割合の誤りについても指摘した。
- 他チームが利用できるワークフローテンプレートを作成し、利用者と基盤担当の責任を明確にした。
- 次回実行で正しい状態になる処理と、対応が必要な異常を区別してワークフロー・通知を整理し、運用負荷を低減した。
- その他: オンライン機械学習基盤、特許データの前処理・検索基盤、データマートの作成・保守。

パーソルキャリア / システムエンジニア

2017年10月–2019年4月

- 求人・転職領域のWebサービスとデータ基盤で、バックエンドの保守・改善を担当。
- 機械学習システム向けオンプレミスHadoop基盤を構築。
- 機械学習プロジェクトの進め方を文書化し、既存の機械学習エンジンを改修・レビュー。
- 事業担当者向けの基礎統計学勉強会を実施。

株式会社IPオンウェブジャパン / データマイニングエンジニア

2016年2月–2017年9月

- 広告配信・収益データの分析と運用支援を担当し、ログ・集計結果から不整合や異常の原因を調査。
- 多腕バンディット法を用いたキャンペーン予算最適化、海外SSPのPrivate Auction導入に従事。
- インフィード広告の配信トラブル改善、代理店向けのパフォーマンス改善コンサルティングを担当。

株式会社Spotlight / エンジニア

2014年4月–2016年1月

- O2O送客サービス「楽天チェック」の開発に従事。
- 社内レポートシステム、テストユーザー機能、大手コンビニチェーン導入向けの機能を開発。
- レポート生成・表示処理などの性能を改善。

学歴・資格・研修

- 早稲田大学大学院 創造理工学研究科 建築学専攻 博士課程単位取得退学。
- IELTS Academic 6.5（2012年11月）。
- 早稲田大学EDGE グローバルアントレプレナー育成プログラム修了（2016年12月）。
- The Fab Academy, Certificate（2013年8月）。
- 早稲田大学博士実践特論カリキュラム修了（2011年9月）。
- Machine Learning, Coursera / Stanford University（2022年2月）。
- Deep Learning, Coursera / Deeplearning.ai（2022年3月）。

受賞

- 日本建築学会関東支部若手優秀研究報告賞（2013年3月）。
- 早稲田大学鶴田奨学金（2013年2月）。

技術記事・翻訳

- 一つのbug対応を伸ばしたら、Software Factoryになった 役割別Agent、独立レビュー、E2E、品質確認、人間の注意力の制約を、約3日間の実践から整理。
- コードだけでなく、つくり方を残す Software Factoryの実行・検証と、個人・組織の学習を区別して考察。
- 生成するルールと、選び抜く目 Xenakisの音楽・建築・数学を題材に、生成条件の設計と成果の評価を考察。
- HTTP/3 explained 日本語版: HTTP/3 QUIC解説の一部を翻訳。
- キャッシュ局所性と性能改善: 最適化が有効な条件を整理。
- SQLAlchemy IdentityMapの実装解析: 同一性・更新・参照の境界条件を調査。
- MySQL全文検索のcommitに関する検証: n-gram検索とテスト時の注意点を整理。
- Digdagのリトライと冪等性: 重複実行の原因と再実行設計を解説。

論文・寄稿

- 「Location Based Social Networkによる都市分析 都市・建築分野でのビッグデータ解析の試み」 2013年度日本建築学会全国大会 情報システム技術部門研究協議会寄稿文。
- “Collective Background Extraction for Station Market Area by using Location Based Social Network” Journal of Civil Engineering and Architecture, 7(3), 282–289, 2013.
- 『スマートライフ』、パレード社、2011年。
- “A New Method for Analyzing the Relationship between City and Human Behavior using Geo-Tagging Social Networking Service” International Union of Architects Academic Program, 10074, 388–393, 2011.
- 「Arduino Pachube SketchUpと連携した建築モニタリングシステムの基礎的研究」 日本建築学会技術報告集、16(33)、791–794、2010年。

作品

- Plant Pod（2013年3月） 植物の生育環境を計測し、その状態を顔文字で表すことで、人が状態を読み取れるようにする作品。
- Live Camera（2010年8月） Webカメラの色彩情報をクラスタリングし、人物の識別を抑えながら研究室の状態とその時系列を可視化。
- Environmental Sensing（2010年3月） 低価格のセンサーモジュールで環境を計測し、サーバーと同期した情報を3D CAD上に表示。